Day(30)

(5.2) Bitwise operators (most imp)

=> The purpose of Bitwise operators is that "to perform various operations on the integer Data in various fields like set operations complex math operations comparisons etc. in the form of Bit by Bit".

=> Bitwise operators are applicable only on integer Data but floating point values bcoz integer Data contains certainity where floating point values does not contain certainity.

=> The Execution process of Bitwise operators is that

- (i) Bitwise operators converts the given integer data into Binary format.
- (ii) Bitwise operators perform operations on Binary Data in the form of Bit by Bit depends on type of Bitwise operator we apply.
- (iii) The Result of Bitwise operators is available internally in the form of Binary and we display the result then it show to end user in the form of Decimal Number system (default number system).

=> Since the Bitwise operators perform the operations in the form of Bit by Bit and Hence they named as Bitwise operators.

=> In Python programming we have 6 types of Bitwise operator.

| SLNo. | SYMBOL | MEANING |
|---|---|---|
| 1. | << | Bitwise left shift operator |
| 2. | >> | Bitwise Right shift operator |
| 3. | & | Bitwise AND operator |
| 4. | \| | Bitwise OR operator |
| 5. | ~ | Bitwise Complement operator |
| 6. | ^ | Bitwise XOR operator |

## (1) Bitwise left shift operator (<<)

Syntax = Given Number << No. of Bits

⇓

=> The execution process of Bitwise leftshift operator (<<) is that "it moves Number of Bits towards left side by Adding Number of zeroes (Number of zeroes = Depending No. of bits we flipped-off) at Right side.

Examples

```
>>> a=10
>>> b = a<<3
>>> print(b) ---80
>>> print(4<<3) ---32
>>> print(9<<2) ---36
>>> print(10<<0) ---10
>>> print(10.3<<2) --- TypeError
```

$4 \times 2^3 = 4 \times 8 = 32$

10<<3
10 x 10

16 bit Register --- a=10.

>>>a=10 ---> | 0000 | 0000 | 0000 | 1010 |

16 bit register --- a=10

>>> b=a<<3 ---> | 0000 | 0000 | 0000 | 1010 |

b ---> | 0000 | 0000 | 0101 | 0000 |

Result is 80

>>> print(b) ---80.

formula:-

varname = Given Number << No. of Bits

varname = Given Number x $2^{No.\ of\ Bits}$.

```
>>> print(2<<3) ---16
>>> print(3<<2) ---12
>>> print(4<<2) ---16.
```

$2 \times 2^3 = 2 \times 8 = 16$

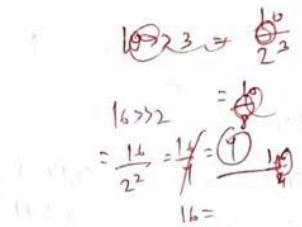
## (11) Bitwise Rightshift operator (>>)

Syntax : `varname = Given Data >> No.of Bits`

=> The Execution process of Bitwise Rightshift operator (>>) is that "it moves number of Bits towards Right side by Adding Number of zeroes (Number of zeroes = Depending No. of bits we flipped - off) at left side.

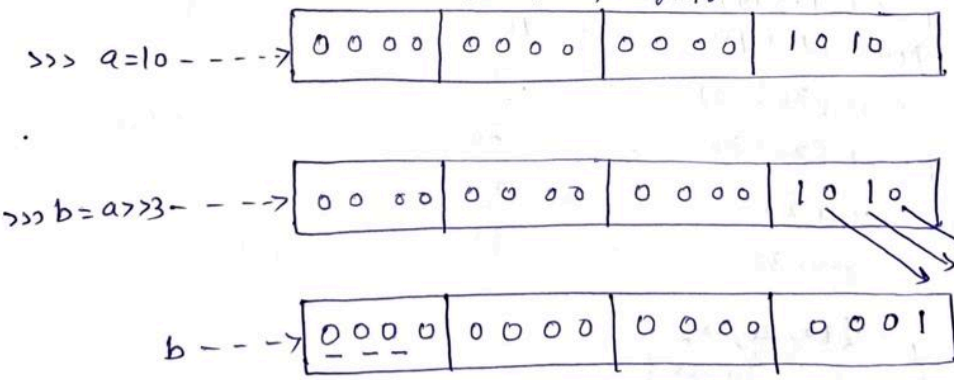
Examples:

```
>>> a=10
>>> b = a>>3
>>> print(b) - - - 1
>>> print (16>>2) - - - 4
>>> print (32>>3) - - - 4
>>> print (32>>0) - - 32
>>> print (80.5<<4) - - - Typeerror
```



Examples

a=10 - 16 Bit Register.




```
>>> print(b) - - -> 1
```

formula: $result = \frac{Given\ Data}{2^{No.of\ Bits}}$

in python ---> Given Data // $2^{No.of\ Bits}$.

```
>>>print(16>>2) - - - 4
>>> print (32>>3) - - - 4.
```

## (iii) Bitwise AND operator (&)

(अगर अगर var 1, 1 → होगी 1 होगा).

syntax :- var1 & var2

=> The functionality of Bitwise AND operator (&) is shown below

| var1 | var2 | var1 & var2 |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 . |

Examples

```
>>> a=4 ------ 0100.
>>> b=3 ------ 0011
>>> c=a&b ---- 0000.
```

```
  0100
And.0011
  0000
```

meaning >>> print(c) --- 0

```
>>> print(10&15) ---- 10.
>>> print(15&10) ---- 10
>>> print(30&20) ---- 20
>>> print(20&30) ---- 20.
>>> 30 and 20 ------ 20
>>> 20 and 30 ------ 30
```

→

```
>>> s1={10,20,30}
>>> s2={15,20,35}
>>> s3=s1&s2
>>> print(s3,type(s3)) ---- {20} <class 'set'>
>>> s1={"Apple","mango"}.
>>> s2={"mango","kiwi"}
>>> s3=s1&s2
>>> print(s3,type(s3)) ---- {'mango'} <class 'set'>
```

```
>>> s1 = {1,2,3.4}
>>> s2 = {4.5, 6.7}
>>> s3 = s1 & s2
>>> print (s3, type(s3))— - - set() <class 'set'>
>>> "Apple" & "mango" — - - Type error
>>> "apple" and "mango" — - - 'mango'
>>> 1.2 & 3.4 — - - Type error
>>> 1.2 and 3.4 — - - 3.4
```

## (IV) Bitwise OR operator (|)

(कोई एक variable भी 1 होगा तो 1 Result होगा)

Syntax : var1 | var2

=> The functionality of Bitwise OR operator (|) is shown below.

| var1 | var2 | var1 \| var2 |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

Examples.

```
>>> a = 4 — - - 0100.
>>> b = 3 — - - - 0011.
>>> c = a|b - - - - 0111 — result 7
>>> a = 5
>>> b = 4
>>> c = a|b
>>> print(c) — - - 5.
>>> print(10|15) — - - 15
>>> print (15|10) — - - 15
>>> print (30|20) — - - 30
>>> print (20|30) — - - 30.
>>> 30 or 20 — - - 30
>>> 20 or 30 — - - 20.
```

most imp

```
>>> s1 = {10, 20, 30}
>>> s2 = {15, 20, 35}
>>> s3 = s1|s2
>>> print(s3, type(s3)) — — — {10, 20, 30, 15, 35} <class 'set'>
>>> s1 = {"Apple", "Mango"}
>>> s2 = {"Mango", "Kiwi"}
>>> s3 = s1|s2
>>> print(s3, type(s3)) — — — {"Apple", 'Mango', Kiwi'} <class 'set'>
```

=

```
>>> s1 = {1.2, 3.4}
>>> s2 = {4.5, 6.7}
>>> s3 = s1|s2
>>> print(s3, type(s3)) — — — {1.2, 3.4, 4.5, 6.7} <class 'set'>
```

=

```
>>> "Apple"|"Mango" — — — Type error.
>>> "apple" or "mango" — — — 'apple'.
>>> 1.2|3.4 — — — Type error
>>> 1.2|3.4 — — — 1.2
                    =.
```

Day(37)

## (X) Bitwise Complement operator (~)

Syntax :- `varname = ~Given Number.`

=> The Execution process of Bitwise complement operator (~) is that "it inverts the given bits".

=> Inverting the bits is nothing but 1 becomes 0 and 0 becomes 1.

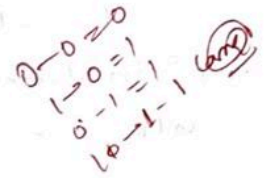
=> The formula for Bitwise complement operator (~) is given below

$$\sim \text{Given Number} = -(\text{Given Number} + 1)$$

=

Example

```
>>> a = 10
>>> print(~a)  - - - - -11
>>> a = 100
>>> print(~a)  - - - - -101
>>> a = 123
>>> print(~a) - - - - - 122
```



Q) Prove that ~10 is -11 (Internal process of)

-ve number = 2's complement

proof:- Given ~10 = -11

Here -11 is the opposite counter part of 11

- Given Number 10 whose Binary part is 1010

=> ~10 - - - - - - ~(1010) = 0101 (which is the Binary form of -11 which is the 2's complement 11).

Note: All the negative numbers are 2's complement of their +ve numbers.

Example: 10 whose counter part is -10 - - - which is 2's complement of 10].

=> Here -11 is the opposite counte part of 11 (2's complement of 11).

=> All negative Number stored in main memory in the form of 2's complement (2's complement = 1's complement +1)

=> Here we take 11 and whose Binary Form is 1011

(1's complement of any Number = 1 becomes 0 & 0 becomes 1)

=> 1's complement of 11 is - - - - 0100

=> 2's complement of 11 is - - - 1's complement of 11+1

=> 0100

=> 0001 [redacted]

=> 0101 - - - which is 2's complement of 11 - - - result is -11

[redacted]

Q. prove that ~16 is -17,

proof:- Given ~16 = -17

Here -17 is the opposite counts part of 17

=> Given Number 16 and whose binary part is 0001 0000

=> ~16 - - - - - -~(0001 0000) = 1110 1111 (which is the binary Form of -17)

=> Here -17 is the opposite counts part of 17

=> All Negative Number stored in main memory in the form 2's complement (2's complement = 1's complement+1)

=> Here we take 17 and whose Binary form is 0001 0001

(1's complement of any number = 1 becomes 0 & 0 becomes 1)

=> 1's complement of 17 is - - - 1110 1110

=> 2's complement of 17 is - - - 1's Complement of 17+1

=> 1110 1110

0000 0001

(Binary Addition rules ( 0+0, 1+0=1, 0+1=1, 1+1=0 with carry 1).

1110 1111

Q. prove that ~15 is -16

proof:- Given ~15 = -16

Here -16 is the opposite counts part of 16

=> Given Number 15 and whose Binary part is 0000 1111

=> ~15 - - - - ~(0000 1111) = 1111 0000

(which is the Binary form of -16).

=> Here -16 is the opposite counter part of 16

=> All Negative Number stored in main memory in the form 2's Complement (2's Complement = 1's Complement +1)

=> Here we take 16 and whose Binary form is 0001 0000

(1's complement of any Number = 1 Becomes 0 & 0 Becomes 1)

=> 1's Complement of 16 is - - - - 1110 1111

=> 2's Complement of 16 is - - - - 1's Complement of 16+1

=> 1110 1111
0002 0001 Binary Addition
( (0+0=0, 1+0=1, 0+1=1, 1+1=0 with carry 1)
1111. 0000 (which is the Binary Form of -16)

(VI) Bitwise XOR operator (^)

* Inside process of Bitwise complement operator :- -(b+1)

>>> a=10 ---> 1010 (Inverting the bits)
>>> b=~a ---> 0101 ---- it is the result of (-11)

Proof:
=> Given Number is 11 --- Binary ----> 1011
=> 1's complement of 11 ---------> 0100 (Inverting the bits)
=> 2's complement of 11 = 1's complement of 11 + 1
= 0100 + 1
= 0100
+ 0001
0101 --- 2's complement of 11 (-11)

(VI) Bitwise XOR operator (^)

syntax: Varname = value1 ^ value2

- The functionality of Bitwise XOR operator (^) is expressed with the following Truth Table.

| value1 | value2 | value1 ^ value2 |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

Bits same value rahega to (0) or bits different rahega (1)

Examples:
>>> print(2^3) ---- 1
>>> print(10^15) ---- 5.

Special points

Static Number different रहा तो ①.

```
>>> s1 = {10, 20, 30}.
>>> s2 = {15, 20, 25}.
>>> s3 = s1.symmetric_difference(s2)
>>> print(s3, type(s3)) - - - - {10, 15, 25, 30} <class 'set'>
```

=

```
>>> s1 = {10, 20, 30}
>>> s2 = {15, 20, 25}
>>> s3 = s1 ^ s2        # Bitwise XOR operator (^)
>>> print(s3, type(s3)) - - - - {10, 15, 25, 30} <class 'set'>.
```

=

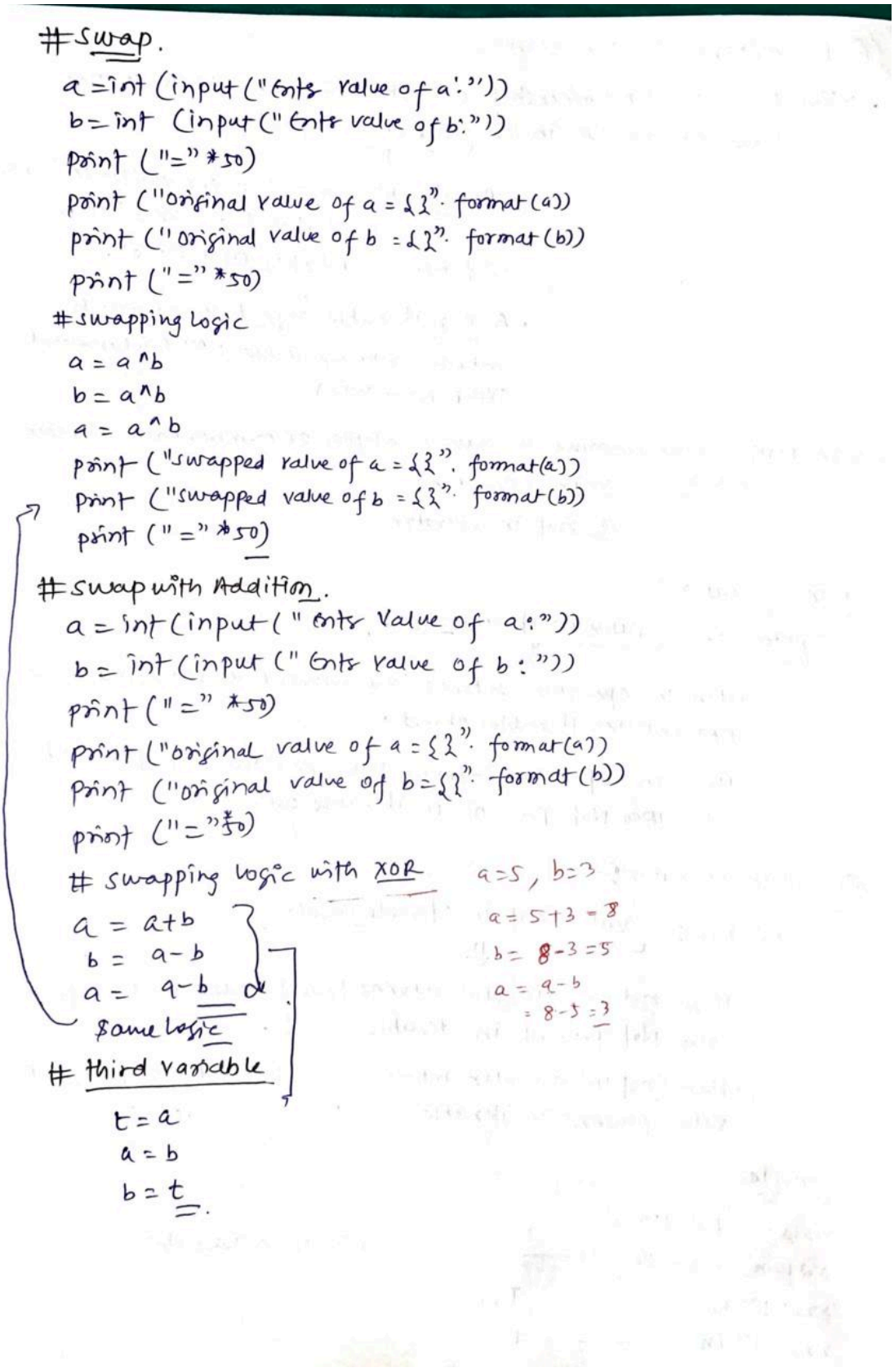
```
>>>s1 = {"Apple", "mango", "kiwi"}
>>> s2 = {"sbbery", "mango", "Guava"}.
>>> s3 = s1^s2
>>> print(s3, type(s3)) - - - - {'guava', 'apple', 'kiwi', 'sberry'}
                                  <class 'set'>
>>> {1.2, 2.3, 3.4} ^ {1.2, 2.3, 4.5} - - {3.4, 4.5}.

>>>> 1.2 ^ 2.3 - - - - Type error.
>>> "apple" ^ 1.2 - - - - Type error.
```

=

Simple logic ---- swapping of two integer values :—

```
>>> a = 3
>>> b = 4
>>> print(a, b) - - - - 3 4
>>> a = a ^ b
>>> b = a ^ b
>>> a = a ^ b
>>> print(a, b) - - - - 4 3
```

=.



```
#swap.
a=int(input("Enter value of a:"))
b=int(input("Enter value of b:"))
print("="*50)
print("original value of a={}".format(a))
print("original value of b={}".format(b))
print("="*50)
#swapping logic
a=a^b
b=a^b
a=a^b
print("swapped value of a={}".format(a))
print("swapped value of b={}".format(b))
print("="*50)
#swap with Addition.
a=int(input("Enter value of a:"))
b=int(input("Enter value of b:"))
print("="*50)
print("original value of a={}".format(a))
print("original value of b={}".format(b))
print("="*50)
# swapping logic with XOR
a=a+b
b=a-b
a=a-b
same logic
#third variable
t=a
a=b
b=t
```

a=5, b=3
a=5+3=8
b=8-3=5
a=a-b
=8-5=3

# 5.Bitwise operators

- in python programming we haave 6 types of bitwise operators:
- 1.Bitwise left shift operator
- 2.Bitwise right shift operator
- 3.Bitwise and operator

- 4.Bitwise or Operator
- 5.Bitwise complement operator
- 6.Bitwise XOR operator

## 1.Bitwise left shift operators:

- syntax:= Given Number< < No.of Bits

In [1]:
```
a=10
b=a<<3
print(b)
```

```
80
```

In [2]:
```
print(4<<3)
```

```
32
```

In [3]:
```
print(9<<2)
```

```
36
```

In [4]:
```
print(10<<0)
```

```
10
```

In [5]:
```
print(10.3<<2)  #
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[5], line 1
----> 1 print(10.3<<2)

TypeError: unsupported operand type(s) for <<: 'float' and 'int'
```

## 2.Bitwise Right shift Operators:

- syntax:= varname=Given data >> no.of Bits

In [7]:
```
a=10
b=a>>3
print(b)
```

```
1
```

In [8]:
```
print(16>>2)
```

```
4
```

In [9]:
```
print(32>>3)
```

```
4
```

In [10]:
```
print(32>>0)
```

```
32
```

## 3.Bitwise And Operator

- syntax:= var1 & var2

In [11]:
```
a=4
b=3
c=a&b
print(c)
```

```
0
```

In [12]:
```
print(10&15)
```

```
10
```

In [13]:
```
print(15&10)
```

```
10
```

In [14]:
```
print(30&20)
```

```
20
```

In [15]:
```
print(20&30)
```

```
20
```

In [16]:
```
30 and 20
```

Out[16]: 20

In [17]:
```
20 and 30
```

Out[17]: 30

In [18]:
```
s1={10,20,30}
s2={15,20,35}
s3=s1&s2
print(s3,type(s3))
```

```
{20} <class 'set'>
```

In [19]:
```
s1={"apple","mango"}
s2={"mango","kiwi"}
s3=s1&s2
print(s3,type(s3))
```

```
{'mango'} <class 'set'>
```

In [20]:
```
s1={1.2,3.4}
s2={4.5,6.7}
s3=s1&s2
print(s3,type(s3))
```

```
set() <class 'set'>
```

In [21]:
```
"Apple"&"mango"
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[21], line 1
----> 1 "Apple"&"mango"

TypeError: unsupported operand type(s) for &: 'str' and 'str'
```

In [22]:
```
"apple" and "mango"
```

Out[22]: 'mango'

In [23]:
```
1.2 and 3.4
```

Out[23]: 3.4

### 4.Bitwise OR operators:-

- syntax:- var1|var2

In [24]:
```
a=4
b=3
c=a|b
print(c,type(c))
```

7 <class 'int'>

In [25]:
```
a=5
b=4
c=a|b
print(c,type(c))
```

5 <class 'int'>

In [26]:
```
print(10|15)
```

15

In [28]:
```
print(15|10)
```

15

In [29]:
```
print(30|20)
```

30

In [30]:
```
print(20|30)
```

30

In [31]:
```
30 or 20
```

Out[31]: 30

In [32]:
```
20 or 30
```

Out[32]: 20

In [33]:
```
s1={10,20,30}
s2={15,20,35}
s3=s1|s2
print(s3,type(s3))
```

{35, 20, 10, 30, 15} <class 'set'>

In [34]:
```
s1={"apple","mango"}
s2={"mango","kiwi"}
s3=s1|s2
print(s3,type(s3))
```

{'mango', 'kiwi', 'apple'} <class 'set'>

In [35]:
```
s1={1.2,3.4}
s2={4.5,6.7}
s3=s1|s2
print(s3,type(s3))
```

```
{1.2, 3.4, 4.5, 6.7} <class 'set'>
```

In [36]:
```
"apple"|"mango"
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[36], line 1
----> 1 "apple"|"mango"

TypeError: unsupported operand type(s) for |: 'str' and 'str'
```

In [37]:
```
"apple" or "messege"
```

Out[37]:
```
'apple'
```

In [38]:
```
1.2|3.4
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[38], line 1
----> 1 1.2|3.4

TypeError: unsupported operand type(s) for |: 'float' and 'float'
```

In [39]:
```
1.2 or 3.4
```

Out[39]:
```
1.2
```

In [41]:
```
a="apple"
b="ball"
c=a|b
print(c)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[41], line 3
      1 a="apple"
      2 b="ball"
----> 3 c=a|b
      4 print(c)

TypeError: unsupported operand type(s) for |: 'str' and 'str'
```

## 5.Bitwise complement operators

- synatx:- varname=~given number

In [42]:
```
a=10
print(~a)
```

```
-11
```

In [43]:
```
a=100
print(~a)
```

```
-101
```

In [45]:
```
a=-123
print(~a)
```

```
122
```

## 6.Bitwise XOR operator

- Syntax:= varname=value1^value2

In [46]:
```
print(2^3)
```

```
1
```

In [47]:
```
print(10^15)
```

```
5
```

In [48]:
```
#special points
s1={10,20,30}
s2={15,20,25}
s3=s1.symmetric_difference(s2)
print(s3,type(s3))
```

```
{10, 15, 25, 30} <class 'set'>
```

In [49]:
```
s1={10,20,30}
s2={15,20,25}
s3=s1^s2
print(s3,type(s3))
```

```
{10, 15, 25, 30} <class 'set'>
```

In [50]:
```
s1={"apple","mango","kiwi"}
s2={"sberry","mango","guava"}
s3=s1^s2
print(s3,type(s3))
```

```
{'sberry', 'apple', 'kiwi', 'guava'} <class 'set'>
```

In [51]:
```
{1.2,2.3,3.4}^{1.2,2.3,4.5}
```

Out[51]:
```
{3.4, 4.5}
```

In [52]:
```
1.2^2.3
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[52], line 1
----> 1 1.2^2.3

TypeError: unsupported operand type(s) for ^: 'float' and 'float'
```

In [53]:
```
#swapping of two integer values:-
a=3
b=4
print(a,b)
a=a^b
b=a^b
```

```
a=a^b
print(a,b)
```

```
3 4
4 3
```

In [54]:
```
#swap
a=int(input("enter value of a:"))
b=int(input("enter value of a:"))
print("="*50)
#swapping logic
a=a^b
b=a^b
a=a^b
print("swapped value of a={}".format(a))
print("swapped value of b={}".format(b))
print("="*50)
```

```
==================================================
swapped value of a=60
swapped value of b=50
==================================================
```

In [56]:
```
#swap with addition
a=int(input("enter value of a:"))
b=int(input("enter value of b:"))
print("="*50)
print("original value of a={}".format(a))
print("original value of b={}".format(b))
print("="*50)
#swapping logic with xor
a=a+b
b=a-b
a=a-b
print("swapped value of a={}".format(a))
print("swapped value of b={}".format(b))
print("="*50)
```

```
==================================================
original value of a=5
original value of b=3
==================================================
swapped value of a=3
swapped value of b=5
==================================================
```

In [ ]:
```

```